

Semester: Semester I, Jan to April 2024

Date: Thursday, April 4, 2024

Duration: 1 week

Weight: 20% **Total Points:** 50

Lecturer(s):

Dr. Nathan Amanquah: namanquah@ashesi.edu.gh

INSTRUCTIONS

Choose one of the options given after the preamble.

Final Project

Objective

The intention is to determine and evaluate the students' ability to create properly structured tasks and a scheduler, use of interrupts, timers, analog (input/output), assembly language programming, communication. Work will be evaluated on ability to demonstrate these features, while maintaining concurrency and responsiveness of the embedded system developed. You will present your work as a demonstration, and a little bit of documentation. The following applies to all options that may be chosen:

- There must be a one-minute video demonstrating and documenting the work done. You may have additional videos if you prefer, but the one-minute one will be graded.
- Your work will be graded by both an in-person demo as well as by your code.

Options are given below.

In all cases, making us of an RTOS earn 2 points extra credit.

Option 1:

Overview: Automation of a robotic vehicle (team of two):

[35 points]

The system to be developed is a remotely operated vehicle. Build a robotic vehicle using the available chassis. Interface the vehicle to one microcontroller which will directly control it. Build a remote controller (another microcontroller based system) which will be used by a human operator to remotely communicate instructions to the vehicle. The robot cart indicates its status on an LCD screen or 7-segment display.

The two microcontroller may communicate using a serial protocol of your choice (e.g. UART [using Bluetooth or cable], SPI, I²C etc). The remote control module has an LCD screen and a keypad for entry of command and display of status messages. There are LEDs and a buzzer that also indicate various conditions as spelt out below. The user makes use of the keypad to move forward, reverse, turn left, right, or stop. When in a move mode, it keeps the mode until instructed otherwise or some conditions make it stop. A pot is used to adjust the speed. Other keys may be used for additional operations as needed (see other operations below)

The robot cart has headlights and tail lights as well as a buzzer. It has an ultrasonic sonic sensor to help detect obstacles around it in a sweeping motion. It has an LCD and a buzzer to indicate status of the cart. (details specified below).

The vehicle should be able to

- i. Move forward and reverse [2]
- ii. Make a turn left or right [2]
- iii. Use a variable resistor/pot to adjust the speed from zero to max speed. You may have 0 to 7 speed levels. (NB: very low “speeds” may not cause movement) [2]
- iv. The robot cart must flash LEDs in the pattern RBBBR while it is powered (think of an emergency services vehicle). Each lamp lights for 200ms and the whole cycle repeats. [2]
- v. When going forwards it lights a RED LED on the tail end, but lights a white one(on the tail end) if reversing. (use the colours you have) [2]
- vi. When going reverse, it beeps at 500ms intervals [2]
- vii. The car must be controllable (eg over cables, or wireless e.g. BT) [2]
- viii. Movement commands must be specified by keypad (or joystick). [2]
- ix. The remote controller should have a flashing pattern continuously while powered. The pattern should be unique and different from that of the cart [2]
- x. The remote controller beeps whenever a command is sent successfully. [2]
- xi. Display feedback & status for humans (eg using LCD/7-seg display) on the cart. [2]
- xii. Sense when the cart is too close to an obstacle, beep 3 times quickly [2]
- xiii. Be able to scan for obstacles in a 180° area in front of the cart [2]
- xiv. Provide a self-test mode where given an appropriate input, the MCU tests/demonstrates each available feature one after the other. E.g. flash lights, beep, display unique message, move, etc etc in a fixed sequence. [2]
- xv. Your code must have at least one function written in assembly language(eg initialize one GPIO, or an interrupt handler etc) [3]
- xvi. You are not permitted to use the SysTick handler for timing. [2]
- xvii. Must use the watchdog timer [2]
- xviii. Creativity: [up to 5]
Provide one or more extra distinguishing maneuver or feature for your vehicle. Eg Make use of some sensor, module, technique, algorithm not previously addressed in class. Using RTOS will earn 2 points

Note that you need to think through the design of your system, especially which pins to use for what.

Responsiveness & concurrency, code structure: 5 points
Overall integration & scope of work: 10 points

Total: 50 points

The head to head race:

Fun activity: Run around an obstacle course in a minimum amount of time (you need to power with a power bank or other battery)

35+15+(extra 5 for extra credit.)

Option 3:

Use an embedded microcontroller to remotely control a robotic arm (subject to availability)

Using potentiometers

- i. Be able to move arm member 1 up and down [4]
- ii. Be able to move arm member 2 forwards and backwards [2]
- iii. Rotate arm base (member 2) clockwise and CCW [2]
- iv. Be able to grip [2]
- v. Have a blue and red flash LEDs in the pattern RBBBR while it is powered (think of an emergency services vehicle). Each lamp lights for 200ms and the whole cycle repeats. [2]
- vi. When arm 2 is going forwards it lights a (different) RED LED, but lights a white one if reversing. (use the colours you have) [2]
- vii. When the base is rotating CCW (reverse), it beeps at 500ms intervals [2]
- viii. Provide a local vs remote switch. In remote mode, you may use a mobile phone app or PC to remotely control the movement of the robotic arm. In local mode, control is by a set of pots. [2]
- ix. Display status of operation of the robotic arm for humans (eg using LCD/7-seg display) [2]
- x. Make use of an ultrasonic sensor suitably placed that can detect obstacles while moving arm member 1. Beep when obstacle is detected and stop motion [2]
- xi. Be able to **memorize** and **repeat** a set of steps. Hint: read the start POT settings for a position and save in an array/variables. (press a button to save them). Next move the robot to the final position and save the new POT settings as well. To play back, push a button, it should cause the robot arm to advance to the initial position, then move to the final position. You may make it repeat. [4]
- xii. Provide a self-test mode where given an appropriate input, the MCU tests/demonstrates each available feature one after the other. E.g. flash lights, beep, display unique message, move, etc etc in a fixed sequence. [2]
- xiii. Your code must have at least one function written in assembly language(eg initialize one GPIO, or an interrupt handler etc) [3]
- xiv. You are not permitted to use the SysTick handler for timing. [2]
- xv. Must use the watchdog timer [2]
- xvi. Creativity: [5]

Provide one or more extra distinguishing maneuver or feature for your robotic arm. Eg Make use of some sensor, module, technique, algorithm not previously addressed in class. Using RTOS will earn 2 points

Note that you need to think through the design of your system, especially which pins to use for what.

Responsiveness & concurrency, code structure: 5 points

Overall integration & scope of work: 10 points

Total: 50 points

The head to head competition:

Fun activity: Pick and place items quickly and accurately preferably automatically.

35+15+(extra 5 for extra credit.)

Option 3:

Open project

The objective is to integrate all the knowledge acquired in one project. This is a team project. Teams of 2. The duration of the project is time constrained. For an open-ended project, these are the minimal expectations:

Create a FRDM-KL25Z controlled system. The following features must be present:

- Keep a heartbeat signal indicting activity (eg an set of LEDs blinking a particular pattern repeatedly to indicate all is well) [2]
- Have a signal to indicate/acknowledge each new operation or receipt of command (eg blink LEDs in a particular pattern whenever a command is received.) [2]
- Use of GPIO (at least 2 other uses not specified here) [4]
- Use of **TPM** (at least 2 different configurations eg PWM, input capture etc) [4]
- Use of an Analog module (ADC/DAC/CMP) [2]
- Use of interrupts. [2]
- Should be controllable either over cable or over wireless [2]
- At least one routine must be written in assembly language [3]
- Provide interaction and feedback for humans (eg LCD/7-seg display etc) [2]
- Provide one extra distinguishing maneuver or feature for your system (a trick interaction feature). [2]
- Provide a self-test mode when given an appropriate input, the MCU tests/demonstrates each available feature one after the other. [2]
- Interfacing to the real world [4]
- You are not permitted to use the SysTick handler for timing. [2]
- Must use the watchdog timer [2]
- Creativity: Provide one or more extra distinguishing maneuver or feature for your embedded system. Eg Make use of some sensor, module, technique, algorithm not previously addressed in class. [5]

35 points

Note that you need to think through the design of your system, especially which pins to use for what.

Responsiveness & concurrency, Code Structure: 5 points

Overall integration & scope of work and completeness: 10 points

For team projects, team member grades will be adjusted up/down based on team members' contribution

Total: 50 points

35+15+(extra 5 for extra credit.)

Project ideas:

- Scoreboard (giant score board)
- Analog clock -with “moving hands”
 - Circular LED clock display
- Digital Clock with temperature display
- Video game
- Conveyor belt based object sorting with color detection
- Home automation/
- home security system with keypad entry
- Music processing/effects
- ~~Data logger eg to gather accelerometer information~~
- Wav player
- Electric vehicle/bike motor speed control
- Speedometer for bike
- Pop up traffic light with
- Spinning display (POV led display)
- Health kiosk
- Water tank level meter (polytank sized) with remote communication.
- Out of office indicator
- ____ Other Embedded systems ____

Discuss these for scope before getting started.

The grading scheme follows the point allocation given in the project description

Below is the sheet for grading

Grading scheme: (option 1)

See mark allocation for option one. Two points for each of the following unless otherwise specified:

Robot car	
GPIO	2
(GPIO)	2
ADC+PWM	2
FSM Task + timer	2
GPIO FW,BK lights	2
Conditional beep rev	2
UART-controllable	2
Keypad	2
Fsm task +timer	2
Beep on success	2
LCD#2	2
Ultrasonic+BEEP	2
PWM servo	2
Self test	2
ASM	3
No systic	2
WDG	2

Responsiveness & concurrency, & code structure

5 points

Overall integration & scope of work:

10 points

Option 2 Rubrics

robotic arm	
ADC+PWM servo	4
ADC+PWM servo	2
ADC+PWM servo	2
PWM	2
FSM Task + timer	2
GPIO FW,BK lights	2
Conditional beep rev	2
UART-controllable	2
LCD#2	2
Ultrasonic+BEEP	2
memorize steps (logic)	4
Self test	2
Asm	3

No systic	2
WDG	2

Option 3 Rubrics

Feature/project	
Heartbeat	
Acknowledgement	
GPIO	
TPM	
ADC/CMP/DAC	
Interrupts	
Serial Comms	
DMA	
ASM	
User interaction	
Trick interaction feature	
Self-test/preset action	
Concurrency	
Real world interfacing	
Creativity	

Responsiveness & concurrency, & code structure
Overall integration & scope of work:

5 points
10 points